

"""

SNS 광고 소비심리 자극 측정 모델 (최종 버전)

계산식: $\text{Score} = \frac{\sum_k (\sum_j s_{jk}) \times \text{synergy}(n_j)}{n} \times 100$

- 가중치: 메타분석 r값 기반 정규화

- 신뢰도 보정: 카테고리별 **baseline** 보정

- 맥락 반영: 방향성 × 강도

- 시너지 보정: 복수 카테고리 동시 탐지 시 보너스

"""

```
import numpy as np
```

```
import pandas as pd
```

```
from dataclasses import dataclass, field
```

```
from typing import List, Dict
```

```
from collections import defaultdict
```

```
from sklearn.model_selection import train_test_split
```

```
import torch
```

```
import torch.nn.functional as F
```

```
# =====
```

```
# 1. 가중치 설정 (선행 메타분석 r값 기반)
```

```
# =====
```

```
R_VALUES = {
```

```
    "권위_신뢰": 0.44, # Pornpitakpan (2004)
```

```
    "사회적_증명": 0.317, # Ao et al. (2023)
```

```
    "사회적_정체성": 0.31, # Aguirre-Rodriguez et al. (2012)
```

```
    "호혜성": 0.30, # Cialdini(1984) + Raghurir(2004) 추정
```

```
    "희소성": 0.25, # Ali et al. (2025)  $\beta \rightarrow r$  변환
```

```
    "긴급성": 0.19, # Ladeira et al. (2023)
```

```
    "가격비교": 0.18, # 이상수 (2023)  $\beta \rightarrow r$  변환
```

```
}
```

```
# 정규화 가중치:  $w_i = r_i / \sum r$ 
```

```
SIGMA_R = sum(R_VALUES.values()) # 1.987
```

```
WEIGHTS = {cat: r / SIGMA_R for cat, r in R_VALUES.items()}
```

```
# =====
```

```
# 2. 맥락 설정 (방향성 × 강도)
```

```
# =====
```

```
# 방향성: 긍정(+1.0) / 중립(+0.5) / 부정(-1.0)
```

```
POLARITY = {
```

```
    "긍정": 1.0,
```

```
    "중립": 0.5,
```

```
    "부정": -1.0,
```

```
}
```

```
# 강도: 강(1.5) / 보통(1.0) / 약(0.5)
```

```
INTENSITY = {
```

```

    "강": 1.5,
    "보통": 1.0,
    "약": 0.5,
}

# =====
# 3. 신뢰도 보정 baseline
#
# 워딩이 명확할수록 높음 (권위·신뢰, 희소성)
# 감성적·추상적 표현일수록 낮음 (의인화, 사회적 정체성)
# =====
CONFIDENCE_BASELINE = {
    "권위_신뢰": 0.90,
    "사회적_증명": 0.85,
    "희소성": 0.85,
    "긴급성": 0.85,
    "가격비교": 0.80,
    "호혜성": 0.75,
    "사회적_정체성": 0.65,
    "의인화": 0.60,
}
MAX_CALIBRATION = 2.0 # 보정 계수 상한선

def calibrate_confidence(category: str, confidence: float) -> float:
    """
    카테고리별 baseline으로 신뢰도 보정

    공식:
        보정 계수 = min(baseline / confidence, MAX_CALIBRATION)
        보정된 신뢰도 = min(confidence × 보정 계수, 1.0)
    """
    if category not in CONFIDENCE_BASELINE:
        return confidence
    baseline = CONFIDENCE_BASELINE[category]
    calibration = min(baseline / confidence, MAX_CALIBRATION)
    return min(confidence * calibration, 1.0)

# =====
# 4. 시너지 보정
#
# 한 문장에서 여러 카테고리가 동시 탐지될 때 보너스 적용
# 근거: 복합 설득 전략은 단일 전략보다 강한 소비심리 자극
# 효과를 유발한다 (Cialdini, 1984)
# =====
def synergy_factor(n: int) -> float:
    """

```

동시 탐지된 카테고리 수에 따른 시너지 보정 계수

n=1 → 1.00 (보정 없음)
n=2 → 1.15 (+15% 보너스)
n=3 → 1.30 (+30% 보너스)
n≥4 → 1.45 (+45% 보너스, 상한선)

"""

```
if n <= 1:
    return 1.00
elif n == 2:
    return 1.15
elif n == 3:
    return 1.30
else:
    return 1.45
```

=====

5. 라벨 설정

=====

CATEGORY_LABEL2IDX = {

"해당없음": 0,
"권위_신뢰": 1,
"사회적_증명": 2,
"사회적_정체성": 3,
"호혜성": 4,
"희소성": 5,
"긴급성": 6,
"가격비교": 7,

}

POLARITY_LABEL2IDX = {"긍정": 0, "중립": 1, "부정": 2}

INTENSITY_LABEL2IDX = {"강": 0, "보통": 1, "약": 2}

CATEGORY_IDX2LABEL = {v: k for k, v in CATEGORY_LABEL2IDX.items()}

POLARITY_IDX2LABEL = {v: k for k, v in POLARITY_LABEL2IDX.items()}

INTENSITY_IDX2LABEL = {v: k for k, v in INTENSITY_LABEL2IDX.items()}

NUM_CATEGORIES = len(CATEGORY_LABEL2IDX) # 8

NUM_POLARITIES = len(POLARITY_LABEL2IDX) # 3

NUM_INTENSITIES = len(INTENSITY_LABEL2IDX) # 3

=====

6. 점수 계산

#

핵심 계산식:

문장 점수 = $w_i \times \hat{c}_j \times p_j \times t_j$

시너지 적용 = 문장 내 카테고리 합계 × synergy_factor(n)

```
# 최종 Score =  $\Sigma(\text{시너지 적용 점수}) / n\_sentences \times 100$ 
# =====
```

```
@dataclass
class SentenceResult:
    """문장 하나의 KoBERT 분류 결과"""
    sentence: str
    category: str # 카테고리
    confidence: float # KoBERT 원본 신뢰도
    calibrated_confidence: float # 보정된 신뢰도
    polarity: str # 방향성
    intensity: str # 강도
    score: float # 문장 기여 점수 (시너지 적용 전)
```

```
def calculate_ad_score(sentence_results: List[SentenceResult]) -> dict:
```

```
    """
    가중치 + 신뢰도 보정 + 맥락 + 시너지 반영 점수 계산
```

```
    최종 계산식:
    문장 점수 =  $w_j \times \hat{c}_j \times p_j \times t_j$ 
    시너지 적용 =  $\Sigma_k(s_{jk}) \times \text{synergy\_factor}(n_j)$ 
    Score =  $\Sigma(\text{시너지 적용 점수}) / n \times 100$ 
    """
```

```
    # 문장별로 카테고리 결과 묶기
    sentence_groups = defaultdict(list)
    for result in sentence_results:
        sentence_groups[result.sentence].append(result)
```

```
    category_scores = {cat: 0.0 for cat in WEIGHTS}
    category_counts = {cat: 0 for cat in WEIGHTS}
    synergy_log = [] # 시너지 적용 내역 로그
    total_score = 0.0
```

```
    for sentence, results in sentence_groups.items():
        # 해당없음 제외한 카테고리 수 계산
        valid = [r for r in results if r.category in WEIGHTS]
        n_cats = len(valid)
        synergy = synergy_factor(n_cats)
```

```
        # 문장 내 카테고리 점수 합산
        sentence_raw = sum(r.score for r in valid)
```

```
        # 시너지 보정 적용
        sentence_final = sentence_raw * synergy
        total_score += sentence_final
```

```
    # 시너지 로그
```

```

if n_cats >= 2:
    synergy_log.append({
        "문장": sentence,
        "탐지카테고리": [r.category for r in valid],
        "카테고리수": n_cats,
        "시너지계수": synergy,
        "원점수": round(sentence_raw, 4),
        "보정점수": round(sentence_final, 4),
    })

# 카테고리별 누적 (시너지 반영)
for r in valid:
    category_scores[r.category] += r.score * synergy
    category_counts[r.category] += 1

n = len(sentence_groups)
final_score = (total_score / n * 100) if n > 0 else 0

return {
    "final_score": round(final_score, 4),
    "raw_score": round(total_score, 4),
    "n_sentences": n,
    "category_scores": {k: round(v, 4) for k, v in category_scores.items()},
    "category_counts": category_counts,
    "synergy_log": synergy_log, # 시너지 적용된 문장 내역
}

# =====
# 7. 라벨링 데이터 → KoBERT 학습용 변환
# =====

def normalize_cols(df):
    return df.rename(columns=lambda c: c.strip()
                      .replace(" ", "_").replace(".", "_"))

def prepare_kobert_dataset(df: pd.DataFrame) -> pd.DataFrame:
    """
    라벨링 완료 데이터 → KoBERT 학습용 포맷 변환

    필요 컬럼:
    text      : 광고 문장
    카테고리  : 권위_신뢰 등 (1 표시)
    방향성    : 긍정/중립/부정
    강도      : 강/보통/약
    """
    cat_cols = [c for c in WEIGHTS.keys() if c in df.columns]

```

```

records = []

for _, row in df.iterrows():
    text = str(row.get("text", "")).strip()
    if not text or text == "nan":
        continue

    # 카테고리 추출 (복수 → 가중치 높은 것 우선)
    matched = [
        cat for cat in cat_cols
        if pd.to_numeric(row.get(cat, 0), errors="coerce") == 1
    ]
    if not matched:
        category = "해당없음"
    elif len(matched) == 1:
        category = matched[0]
    else:
        category = max(matched, key=lambda x: WEIGHTS.get(x, 0))

    # 방향성 (없으면 기본값 긍정)
    polarity = str(row.get("방향성", "긍정")).strip()
    if polarity not in POLARITY_LABEL2IDX:
        polarity = "긍정"

    # 강도 (없으면 기본값 보통)
    intensity = str(row.get("강도", "보통")).strip()
    if intensity not in INTENSITY_LABEL2IDX:
        intensity = "보통"

    records.append({
        "text": text,
        "category_label": CATEGORY_LABEL2IDX[category],
        "category_name": category,
        "polarity_label": POLARITY_LABEL2IDX[polarity],
        "polarity_name": polarity,
        "intensity_label": INTENSITY_LABEL2IDX[intensity],
        "intensity_name": intensity,
    })

return pd.DataFrame(records)

```

```

# =====
# 8. KoBERT 멀티태스크 모델
# 카테고리 + 방향성 + 강도 동시 분류
# =====

```

```

class MultiTaskModel(torch.nn.Module):

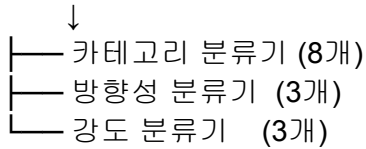
```

"""

카테고리 / 방향성 / 강도 동시 분류 멀티태스크 모델

구조:

BERT 인코더 (공유)



"""

```
def __init__(self, model_name: str):
    super().__init__()
    from transformers import AutoModel
    self.encoder = AutoModel.from_pretrained(model_name)
    hidden = self.encoder.config.hidden_size # 768

    self.category_head = torch.nn.Linear(hidden, NUM_CATEGORIES)
    self.polarity_head = torch.nn.Linear(hidden, NUM_POLARITIES)
    self.intensity_head = torch.nn.Linear(hidden, NUM_INTENSITIES)

    def forward(self, input_ids, attention_mask, token_type_ids=None):
        outputs = self.encoder(
            input_ids=input_ids,
            attention_mask=attention_mask,
            token_type_ids=token_type_ids,
        )
        cls = outputs.last_hidden_state[:, 0, :] # CLS 토큰
        return {
            "category_logits": self.category_head(cls),
            "polarity_logits": self.polarity_head(cls),
            "intensity_logits": self.intensity_head(cls),
        }
```

카테고리 클래스 가중치 (r값 기반, 데이터 불균형 보정)

```
CATEGORY_CLASS_WEIGHTS = torch.tensor([
    0.1,
    WEIGHTS["권위_신뢰"],
    WEIGHTS["사회적_증명"],
    WEIGHTS["사회적_정체성"],
    WEIGHTS["호혜성"],
    WEIGHTS["희소성"],
    WEIGHTS["긴급성"],
    WEIGHTS["가격비교"],
], dtype=torch.float)
```

```
def multitask_loss(outputs, batch):
```

```
"""
```

```
멀티태스크 손실 함수
```

```
총 손실 = 0.6×카테고리 + 0.2×방향성 + 0.2×강도
```

```
"""
```

```
cat_loss = torch.nn.CrossEntropyLoss(  
    weight=CATEGORY_CLASS_WEIGHTS.to(outputs["category_logits"].device)  
) (outputs["category_logits"], batch["category_label"])
```

```
pol_loss = torch.nn.CrossEntropyLoss()(  
    outputs["polarity_logits"], batch["polarity_label"]  
)
```

```
int_loss = torch.nn.CrossEntropyLoss()(  
    outputs["intensity_logits"], batch["intensity_label"]  
)
```

```
# 카테고리 분류에 더 높은 가중치
```

```
return 0.6 * cat_loss + 0.2 * pol_loss + 0.2 * int_loss
```

```
def train_kobert(train_df, val_df, model_name="klue/bert-base"):
```

```
    """KoBERT 파인튜닝 실행"""
```

```
    from transformers import AutoTokenizer, TrainingArguments, Trainer
```

```
    from datasets import Dataset
```

```
    tokenizer = AutoTokenizer.from_pretrained(model_name)
```

```
    model = MultiTaskModel(model_name)
```

```
    def tokenize(batch):
```

```
        return tokenizer(  
            batch["text"],  
            padding="max_length",  
            truncation=True,  
            max_length=128,  
        )
```

```
    train_ds = Dataset.from_pandas(train_df).map(tokenize, batched=True)
```

```
    val_ds = Dataset.from_pandas(val_df).map(tokenize, batched=True)
```

```
    training_args = TrainingArguments(  
        output_dir="/kobert_ad_classifier",  
        num_train_epochs=5,  
        per_device_train_batch_size=16,  
        per_device_eval_batch_size=32,  
        learning_rate=2e-5,  
        warmup_ratio=0.1,  
        eval_strategy="epoch",  
        save_strategy="epoch",
```

```

        load_best_model_at_end=True,
        metric_for_best_model="f1",
    )

def compute_metrics(eval_pred):
    from sklearn.metrics import f1_score, accuracy_score
    preds = np.argmax(eval_pred.predictions, axis=-1)
    return {
        "accuracy": accuracy_score(eval_pred.label_ids, preds),
        "f1": f1_score(eval_pred.label_ids, preds, average="weighted"),
    }

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_ds,
    eval_dataset=val_ds,
    compute_metrics=compute_metrics,
)
trainer.train()
return model, tokenizer

```

```

# =====
# 9. 추론 파이프라인
# =====

```

```
def predict_and_score(ad_text: str, model, tokenizer) -> dict:
```

```
    """
```

광고 텍스트 → 최종 소비심리 자극 점수 출력

Pipeline:

1. 문장 분리
2. KoBERT: 카테고리 + 방향성 + 강도 동시 분류
3. 신뢰도 보정 (카테고리별 **baseline**)
4. 문장 점수 = $w \times \hat{c} \times \text{polarity} \times \text{intensity}$
5. 시너지 보정 (복수 카테고리 동시 탐지 시)
6. 최종 점수 정규화 후 출력

```
    """
```

```
import re
```

```
sentences = [s.strip() for s in re.split(r"[!?\n]", ad_text) if s.strip()]
```

```
sentence_results = []
```

```
model.eval()
```

```
for sentence in sentences:
```

```
    inputs = tokenizer(
```

```
        sentence,
```

```

        return_tensors="pt",
        padding=True,
        truncation=True,
        max_length=128,
    )

    with torch.no_grad():
        outputs = model(
            input_ids=inputs["input_ids"],
            attention_mask=inputs["attention_mask"],
        )

    # 카테고리 분류
    cat_probs = F.softmax(outputs["category_logits"], dim=-1)[0]
    cat_idx = cat_probs.argmax().item()
    confidence = cat_probs[cat_idx].item()
    category = CATEGORY_IDX2LABEL[cat_idx]

    # 방향성
    pol_idx = F.softmax(outputs["polarity_logits"], dim=-1)[0].argmax().item()
    polarity = POLARITY_IDX2LABEL[pol_idx]

    # 강도
    int_idx = F.softmax(outputs["intensity_logits"], dim=-1)[0].argmax().item()
    intensity = INTENSITY_IDX2LABEL[int_idx]

    # 신뢰도 보정
    calibrated = calibrate_confidence(category, confidence)

    # 문장 점수 (시너지 적용 전)
    w = WEIGHTS.get(category, 0)
    pol_v = POLARITY.get(polarity, 0.5)
    int_v = INTENSITY.get(intensity, 1.0)
    score = w * calibrated * pol_v * int_v

    sentence_results.append(SentenceResult(
        sentence=sentence,
        category=category,
        confidence=confidence,
        calibrated_confidence=calibrated,
        polarity=polarity,
        intensity=intensity,
        score=score,
    ))

# 시너지 포함 최종 점수 계산
result = calculate_ad_score(sentence_results)

```

```

# 문장별 상세 결과
result["sentences"] = [
    {
        "문장": r.sentence,
        "카테고리": r.category,
        "원본신뢰도": round(r.confidence, 3),
        "보정신뢰도": round(r.calibrated_confidence, 3),
        "방향성": r.polarity,
        "강도": r.intensity,
        "점수": round(r.score, 4),
    }
    for r in sentence_results
]

return result

# =====
# 10. 실행 예시
# =====
if __name__ == "__main__":

    # 가중치 출력
    print("=" * 60)
    print(f"카테고리별 가중치 ( $\Sigma r = \{SIGMA\_R:.3f\}$ )")
    print("=" * 60)
    for cat, r in sorted(R_VALUES.items(), key=lambda x: -x[1]):
        w = WEIGHTS[cat]
        b = CONFIDENCE_BASELINE.get(cat, "-")
        print(f" {cat:<14} r={r:.3f} 가중치={w*100:.1f}% baseline={b}")

    print("\n" + "=" * 60)
    print("시너지 보정 계수")
    print("=" * 60)
    for n in range(1, 5):
        label = f"n={n}" if n < 4 else "n≥4"
        print(f" 카테고리 {label}개 → ×{synergy_factor(n):.2f}")

    # 수동 테스트 예시
    print("\n" + "=" * 60)
    print("계산 예시")
    print("=" * 60)

    test_sentences = [
        # (문장, 카테고리, 원본신뢰도, 방향성, 강도)
        ("피부과 전문의 강력 추천!", "권위_신뢰", 0.95, "긍정", "강"),
        ("선착순 100명 한정!", "희소성", 0.92, "긍정", "강"),
        # 시너지 테스트: 한 문장에 2개 카테고리

```

```

("오늘까지만 선착순 100명 한정!", "희소성", 0.90, "긍정", "강"),
("오늘까지만 선착순 100명 한정!", "긴급성", 0.88, "긍정", "강"),
# 시너지 테스트: 한 문장에 3개 카테고리
("오늘까지 선착순 50% 할인!", "긴급성", 0.88, "긍정", "강"),
("오늘까지 선착순 50% 할인!", "희소성", 0.85, "긍정", "강"),
("오늘까지 선착순 50% 할인!", "가격비교", 0.82, "긍정", "강"),
]

results = []
for sentence, category, confidence, polarity, intensity in test_sentences:
    calibrated = calibrate_confidence(category, confidence)
    w = WEIGHTS.get(category, 0)
    pol_v = POLARITY.get(polarity, 0.5)
    int_v = INTENSITY.get(intensity, 1.0)
    score = w * calibrated * pol_v * int_v
    results.append(SentenceResult(
        sentence=sentence,
        category=category,
        confidence=confidence,
        calibrated_confidence=calibrated,
        polarity=polarity,
        intensity=intensity,
        score=score,
    ))

final = calculate_ad_score(results)

print(f"\n{'문장':<25} {'카테고리':<14} {'보정신뢰도':>8} {'방향':>5} {'강도':>5} {'점수':>8}")
print("-" * 75)
for r in results:
    print(f" {r.sentence[:23]:<23} {r.category:<14} "
          f"{r.calibrated_confidence:>8.3f} {r.polarity:>5} "
          f"{r.intensity:>5} {r.score:>8.4f}")

print(f"\n[시너지 적용 내역]")
if final["synergy_log"]:
    for log in final["synergy_log"]:
        cats = ", ".join(log["탐지카테고리"])
        print(f" {log['문장'][:30]}")
        print(f" → {cats} ({log['카테고리수']}개) × {log['시너지계수']} "
              f"f= {log['원점수']:.4f} → {log['보정점수']:.4f}")
else:
    print(" 없음")

print(f"\n최종 점수: {final['final_score']}점 / 100점")

# 라벨링 파일 로드 및 학습
print("\n" + "=" * 60)

```

```

print("KoBERT 학습 실행")
print("=" * 60)

df_sy = normalize_cols(pd.read_excel("이서영_직접체크용.xlsx", sheet_name="라벨링
체크"))
df_dh = normalize_cols(pd.read_excel("김도훈.xlsx", sheet_name="라벨링"))
df_mj = normalize_cols(pd.read_excel("김민정.xlsx"))
df_sm = normalize_cols(pd.read_csv("안설민_라벨링_완.csv", skiprows=2))

df_all = pd.concat([df_sy, df_dh, df_mj, df_sm], ignore_index=True)
kobert_df = prepare_kobert_dataset(df_all)

print("라벨 분포:")
print(kobert_df["category_name"].value_counts())

train_df, val_df = train_test_split(
    kobert_df, test_size=0.2, random_state=42,
    stratify=kobert_df["category_label"]
)
print(f"\n학습: {len(train_df)}개 / 검증: {len(val_df)}개")

model, tokenizer = train_kobert(train_df, val_df)

result = predict_and_score(
    "피부과 전문의 추천! 오늘까지만 선착순 100명 50% 할인.",
    model, tokenizer
)
print(f"\n최종 점수: {result['final_score']}점 / 100점")
if result["synergy_log"]:
    print("시너지 적용:")
    for log in result["synergy_log"]:
        print(f" {log['문장']} → ×{log['시너지계수']}")

```